EECS 388

Introduction to Computer Security

Lecture 2:

Message Integrity

September 3, 2026
Prof. Halderman



Course Communication and Resources

Website: EECS388.org
overview, schedule, assignments

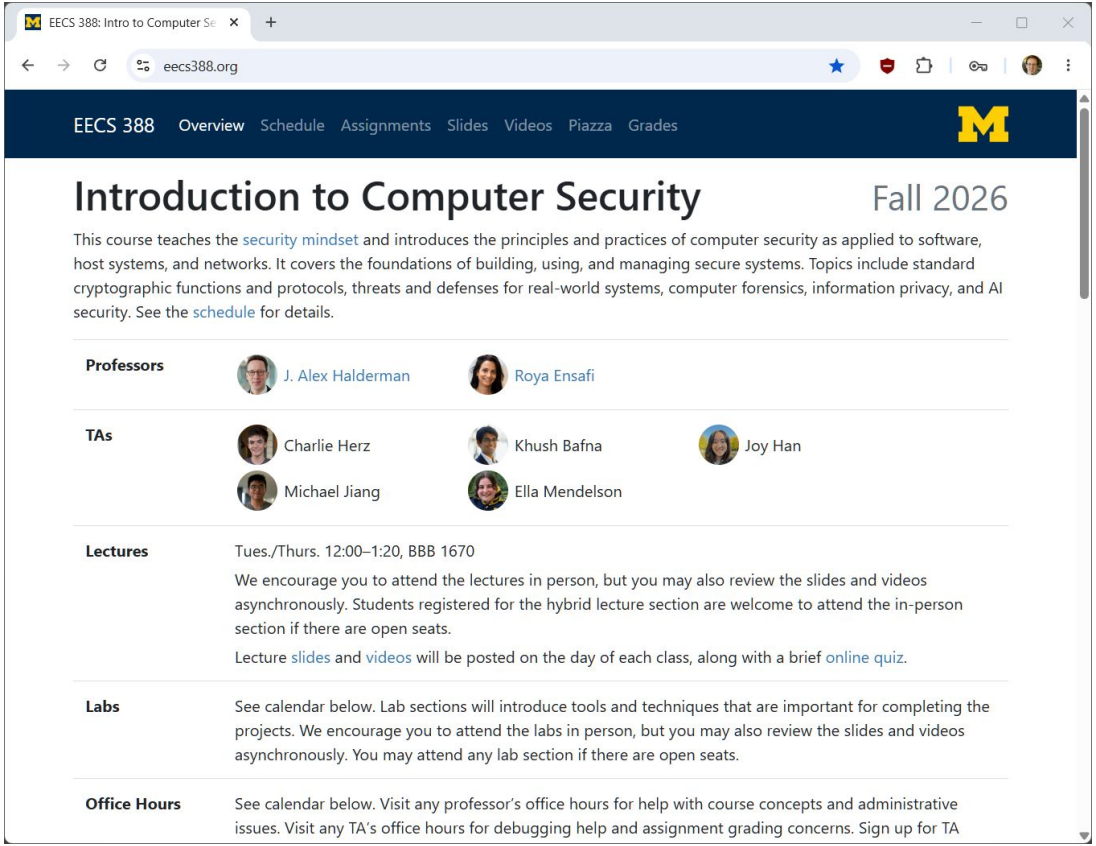
Piazza
questions, discussion, slides, announcements

Canvas
gradebook, lecture videos

Autograder.io
assignment submission/grading

Office Hours
(see calendar on course website)

EECS388-staff@umich.edu
administrative issues



Ethics, Law, and University Policies

Don't be evil!

- Learn to use your security skills responsibly and for good.
- **388 ethics policy:** Respect privacy and property rights **or fail the course.**
- Stay safe: practice attacks **only on systems we give you to test.**

Warning

Federal and state laws criminalize computer intrusion, wiretapping.

- Example: *Computer Fraud and Abuse Act* (CFAA)
- You can be sued or go to prison.

University policies prohibit tampering with campus systems.

- You can be disciplined, even expelled.

Homework: Getting to Know You

Assignment:

Take a selfie and mail it to us
(follow the template shown at right)

Due <u>before next lecture</u>

New Message

Recipients **eecs388-photos@umich.edu**

Subject ***[Your Uniqname]***

> What name should we call you?

> What's your year and major?

> What would you like to learn in 388?

> **Selfie:**



Send

Cryptography

Cryptography

(From Greek: *kryptós+graphein*, "secret writing")

The study of techniques for communicating securely in the presence of an adversary

***Related*: Cryptanalysis**

the study of techniques for *breaking* cryptosystems

Goals for 388: Learn how to safely use **crypto primitives** as building blocks for security

Security properties we'll try to achieve:

Confidentiality
Message Integrity
Sender Authenticity



Cryptographic Theory

Beautiful, highly rigorous

Proofs based on computational complexity

Cryptographic Practice

Delicate engineering

Assumptions based on empirical experience

This course gives only a brief (**and non-rigorous**) overview. Consider EECS 475 or 575 to dive deeper into cryptography!

Goal: Message Integrity

Message integrity ensures that attackers cannot modify messages *without being detected* (they can still do plenty of other bad stuff!)

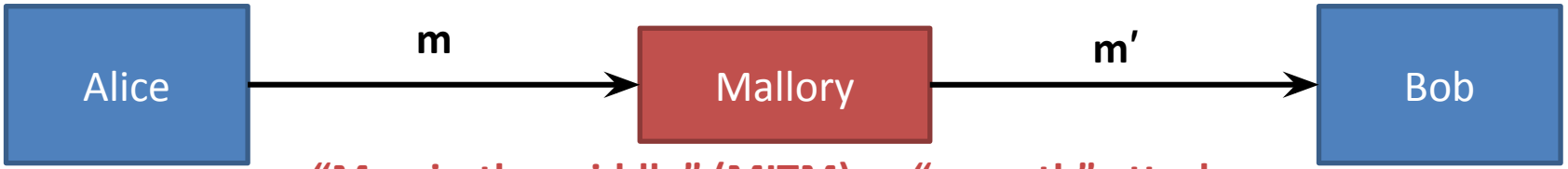
Message integrity is often *even more important* than confidentiality [Why?]

Alice wants to send message ***m*** to **Bob**

- They don't fully trust the messenger (or network) carrying the message
- They want to be sure what Bob receives is what Alice actually sent (**m′** = **m**)

Threat model:

- Mallory can see/modify/forge messages
- Mallory wants to trick Bob into accepting a message Alice didn't send



"Man-in-the-middle" (MITM) or "on-path" attacker

a kind of **active attacker**

Approach: Message Verifier

1. Alice computes *verifier* **v** := ***f***(**m**)
2. 

   e.g. "Attack at dawn", 628369867…

3. Bob verifies that **v′** = ***f***(**m′**), accepts message if and only if this is true

Properties we want for ***f***() ?

- Easily computable by Alice and Bob, but ***not*** easily computable by Mallory
- We lose the game if Mallory can deduce ***f***(**x**) for any **x** ≠ **m**

Idea: Secret ***f***() only Alice and Bob know

Candidate ***f***(): **Random function (RF)**

Input: Fixed size (length of longest **m**)

Output: Fixed size (say, 256 bits)

Construct a giant lookup table by flipping coins for every possible input

0	→	0011111001010…
1	→	1110011010001…
2	→	0101010001010…
…		

Pro: ***Provably secure***

[Show Mallory can't do better than guessing]

Con: ***Completely impractical!***

[Estimate how much storage it would require]

Pseudorandom Functions

Want a function that's practical but "looks random"...

Pseudorandom function (PRF)

Let's build a PRF:

Start with a family of $2^{\mathbf{n}}$ functions
$f_0(), f_1(), f_2(), \ldots, f_{(2\text{\^{}}\mathbf{n})-1}()$
all ***known to Mallory.***

Let our verification function $v() := f_k()$
where **k** is a secret **n**-bit index ("**key**")
known only to Alice and Bob.

What makes for a suitable function family $f()$?

Security definition: A game against Mallory—

1. Choose a secret **k** and an RF $g()$
2. We flip a coin secretly to get bit **b**
3. If **b**=0, let $h() := g()$
   If **b**=1, let $h() := f_k()$
4. Mallory chooses **x**; we announce $h(\mathbf{x})$.
   Repeat step 4 as often as Mallory likes
5. Mallory guesses **b** *in polynomial time**

We say $f()$ is a **secure PRF** if Mallory can't do meaningfully better than random guessing.

* **Note the reliance on computational complexity. Mallory can always win slowly!**
With RF, Mallory **can't possibly** learn unseen outputs.
With PRF, M. can, but at *impractical* (exponential) cost by mounting a "**brute force attack**" on **k**. [Explain?]

Using a PRF for Message Integrity

1. Let $f()$ be a secure PRF (known to everyone)
2. In advance, choose a random key **k** known to Alice and Bob but <u>not</u> Mallory
3. Alice computes $\mathbf{v} := f_k(\mathbf{m})$
4. 

5. Bob verifies that $\mathbf{v'} = f_k(\mathbf{m'})$, accepts message if and only if they are equal

If Bob accepts **m'**, then, with very high confidence, **m'** is identical to **m**. (How high? $1-1/2^n$) [Important assumptions?]

What if Alice and Bob want to send more than one message? [Attacks?] [Solutions?]

This approach follows **Kerckhoffs's Principle**:
"A cryptosystem should remain secure even if attackers know ***everything but the key.***" [Why?]

Annoying question:
Do PRFs *actually exist*?

Annoying answer:
We don't know. (Would imply P≠NP!)

Best we can do:
Use well studied functions where we haven't spotted a problem yet

Cryptographic Hashes

First step towards a practical solution…

Cryptographic Hash Function

Fixed function ***H***(). ***No key*!**

Input: arbitrary length data

Output: fixed size *digest* (**n** bits)

Properties of strong hash functions:

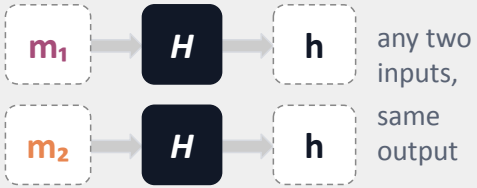
Preimage resistance



For a given output **h**,
hard to find any input **m**
such that **h** = ***H***(**m**).

Hard to reverse a particular output.

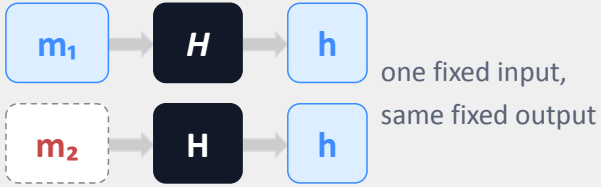
Collision resistance



No value is fixed.
Hard to find *any* pair of inputs
m_1,m_2 such that ***H***(m_1)=***H***(m_2).

Hard to make any two inputs collide.

Second-preimage resistance



For a given input m_1,
hard to find $m_2 \neq m_1$
such that ***H***(m_1)=***H***(m_2).

Hard to match a particular output.

Cryptographic Hashes

First step towards a practical solution…

Cryptographic Hash Function

Fixed function ***H***(). ***No key*!**

Input: arbitrary length data

Output: fixed size *digest* (**n** bits)

Note: There must exist an ***infinite*** number of collisions. [Why?]
"Hard to find" means *computing them* is intractable.

Properties of strong hash functions:

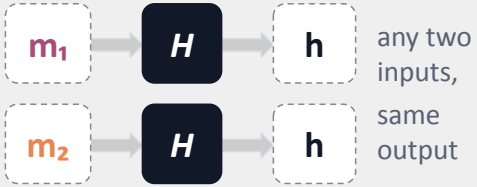
Preimage resistance



For a given output **h**,
hard to find any input **m**
such that **h** = ***H***(**m**).

Hard to reverse a particular output.

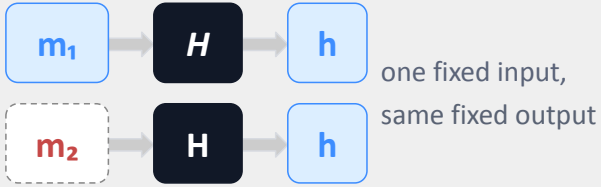
Collision resistance



No value is fixed.
Hard to find *any* pair of inputs
m_1,m_2 such that ***H***(m_1)=***H***(m_2).

Hard to make any two inputs collide.

Second-preimage resistance



For a given input m_1,
hard to find $m_2 \neq m_1$
such that ***H***(m_1)=***H***(m_2).

Hard to match a particular output.

Cryptographic Hashes

First step towards a practical solution…

Cryptographic Hash Function

Fixed function ***H***(). ***No key*!**

Input: arbitrary length data

Output: fixed size *digest* (**n** bits)

Computing hashes with **OpenSSL:**

```
$ echo "hello world 0" | openssl dgst -sha256
8bfa9a398e97152beaaf385847808ad2d828c1c7251f1a45bc7697723827e7e7
$ echo "hello world 1" | openssl dgst -sha256
07bafbe63a0e7c57c572aedf1c228022b537e28785013d7be017fc78731a8cc5
```

Change even a single bit of input, the output looks totally unrelated.

Properties of strong hash functions:

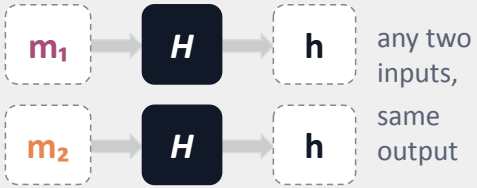
Preimage resistance



For a given output **h**,
hard to find any input **m**
such that **h** = ***H***(**m**).

Hard to reverse a particular output.

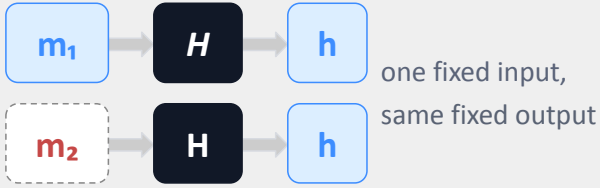
Collision resistance



No value is fixed.
Hard to find *any* pair of inputs
m_1,m_2 such that ***H***(m_1)=***H***(m_2).

Hard to make any two inputs collide.

Second-preimage resistance



For a given input m_1,
hard to find $m_2 \neq m_1$
such that ***H***(m_1)=***H***(m_2).

Hard to match a particular output.

Cryptographic Hashes

First step towards a practical solution…

Cryptographic Hash Function

Fixed function ***H***(). ***No key*!**

Input: arbitrary length data

Output: fixed size *digest* (**n** bits)

Annoying question:

Are existing hashes *actually strong*?

Annoying answer:

We don't know.

Candidates: ~~MD5~~, ~~SHA-1~~, **SHA-256**, SHA-512, SHA-3

Properties of strong hash functions:

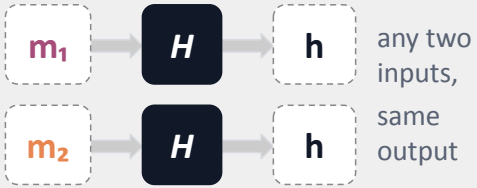
Preimage resistance



For a given output **h**,
hard to find any input **m**
such that **h** = ***H***(**m**).

Hard to reverse a particular output.

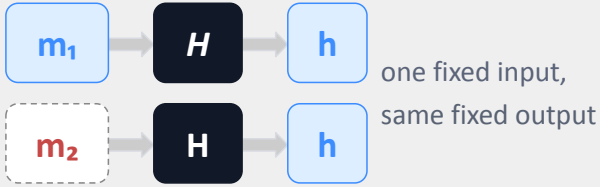
Collision resistance



No value is fixed.
Hard to find *any* pair of inputs
m_1,m_2 such that ***H***(m_1)=***H***(m_2).

Hard to make any two inputs collide.

Second-preimage resistance



For a given input m_1,
hard to find $m_2 \neq m_1$
such that ***H***(m_1)=***H***(m_2).

Hard to match a particular output.

Hashes with Broken Collision Resistance

MD5

Once ubiquitous … broken in 2004

Now it's **easy to find collisions**
(pairs of messages with the same MD5 hash)

Exploited to attack real systems

You'll do this in Project 1!

```
$ echo
d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325 71415a085125e8f7cdc99fd91dbdf280373c5b
d8823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70
| xxd -r -p | openssl dgst -md5
79054025255fb1a26e4bc422aef54eb4

$ echo
d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5b
d8823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70
| xxd -r -p | openssl dgst -md5
79054025255fb1a26e4bc422aef54eb4
```

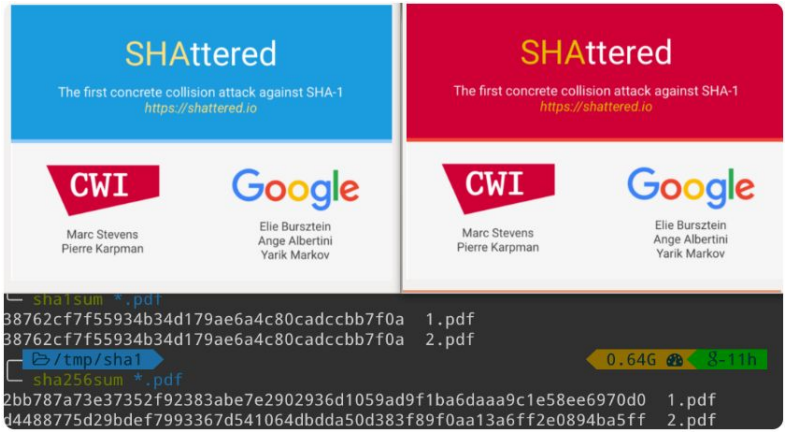
SHA-1

Once ubiquitous … broken in 2017

Rapidly being phased out

Computing first collision cost >$100,000
Today, with improvements <$10,000

Expect attacks will keep getting better!



Constructing SHA-256

SHA-256 is a widely used hash function that is currently thought to be strong

Input: arbitrary length data
Output: 256-bit digest

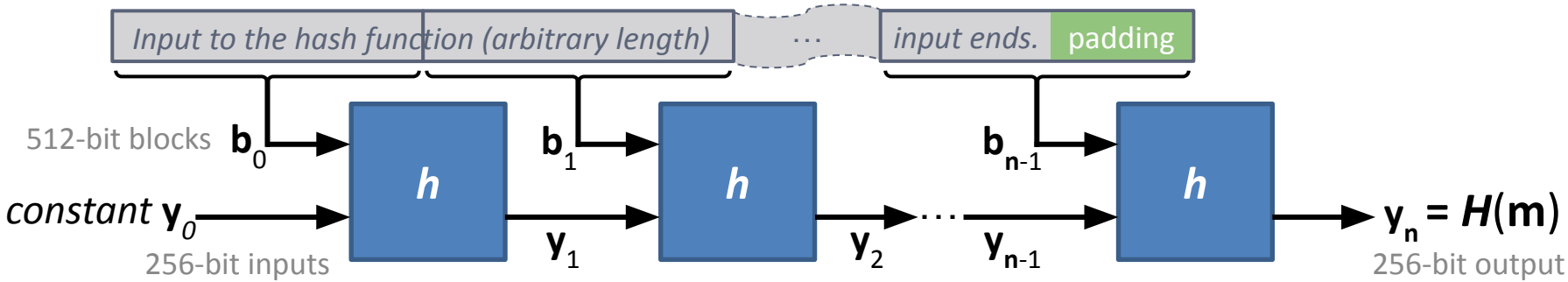
Built from a **compression function *h***:

Inputs: (256 bits, 512 bits), **Output:** 256 bits

reduces 768 bits to 256 bits using a complicated internal function (details out of scope for 388)

Uses the **Merkle–Damgård (MD) construction** (illustrated below) to accept arbitrary-length input by repeatedly applying ***h***():

1. **Pad** the input **m** to next multiple of 512 bits (adds at least 1 bit, uses fixed algorithm [why?]) and split into 512-bit blocks: $\mathbf{b}_0, \mathbf{b}_1, \ldots \mathbf{b}_{n-1}$
2. $\mathbf{y}_0$:= *<256-bit constant>*
   $\mathbf{y}_1 := \boldsymbol{h}(\mathbf{y}_0, \mathbf{b}_0)$ … $\mathbf{y}_i := \boldsymbol{h}(\mathbf{y}_{i-1}, \mathbf{b}_{i-1})$
3. Return $\mathbf{y}_n$ *which is defined to be* **SHA-256**(**m**)



Coming Up

Reminders:

Send selfies before next lecture

Quiz on Canvas after every lecture

Lab Assignment 1 available, due next Thursday at 6pm

Project 1 available, Part 1 due Sept. 17 at 6pm

Tuesday

Randomness and Pseudorandomness

Generating randomness,
PRGs, one-time pads

Thursday

Confidentiality

Simple ciphers,
AES,
block cipher modes